



COMSATS UNIVERSITY ISLAMABAD
Vehari Campus

SENIOR DESIGN PROJECT REPORT
Machine Learning-Based Anomaly Detection
for Distributed Web Application Logs

Senior Design Project I & II

Bachelor of Science in Computer Science

Student: Abu Huraira

Registration No.: CII/TFA18-BCS-125/VHR

Degree: Bachelor of Science in Computer Science

Department: Department of Computer Science

Recorded Capstone: Senior Design Project I (2 CH) + II (4 CH)

Senior Design Project I & II
Fall 2021 - Spring 2022 academic capstone sequence

The academic transcript records Senior Design Project I in Fall 2021 for 2 credit hours and Senior Design Project II in Spring 2022 for 4 credit hours. Together, the two courses form the two-semester capstone sequence documented in this report.

This Senior Design Project investigates machine-learning-based anomaly detection for distributed web application logs. The work combines structured telemetry, feature engineering, unsupervised learning, threshold selection, and a service-oriented implementation suitable for modern web applications.

The project was completed as part of Senior Design Project I and Senior Design Project II in the BS Computer Science curriculum. This report documents the problem definition, design, implementation methodology, evaluation approach, and conclusions of that capstone work.

Abstract

Distributed web applications produce large volumes of logs from APIs, authentication systems, databases, background jobs, and third-party dependencies. Manual inspection is slow and fixed threshold rules can miss combinations of weak operational signals. This project report presents a machine-learning pipeline that converts structured application events into window-based operational features and ranks unusual windows for engineering review.

The design uses eight core features: request count, error rate, average response duration, 95th-percentile response duration, authentication-failure rate, dependency-failure count, retry count, and endpoint entropy. Three unsupervised methods - Isolation Forest, Local Outlier Factor, and One-Class Support Vector Machine - are compared with a supervised Random Forest reference. An ASP.NET Core application layer produces structured events, a feature service aggregates data, and a Python scoring service exposes a model-agnostic internal API. PostgreSQL and OpenSearch are used as representative persistence and investigation technologies.

The benchmark demonstrates that validation-based threshold selection can materially change precision, recall, F1-score, and false-positive rate. The strongest one-class operating points in the benchmark test were produced by Local Outlier Factor and One-Class SVM, while Isolation Forest served as a transparent baseline. The report concludes that anomaly detection should be treated as decision support: model scores must be paired with event traceability, threshold governance, versioned artifacts, security controls, and human review.

Keywords: anomaly detection, machine learning, distributed systems, application logs, observability, ASP.NET Core, One-Class SVM, Local Outlier Factor, Isolation Forest.

Acknowledgments

This report acknowledges the academic foundation provided by the Bachelor of Science in Computer Science curriculum at COMSATS University Islamabad, Vehari Campus. The student's transcript records coursework in programming, data structures and algorithms, databases, operating systems, computer architecture, networks, statistics and probability, numerical computing, and artificial intelligence, together with the two-part Senior Design Project sequence.

List of Abbreviations

API	Application Programming Interface
AUC	Area Under the Curve
CI/CD	Continuous Integration / Continuous Delivery
ERD	Entity-Relationship Diagram
F1	Harmonic mean of precision and recall

FPR	False-Positive Rate
HTTP	Hypertext Transfer Protocol
IF	Isolation Forest
JWT	JSON Web Token
LOF	Local Outlier Factor
ML	Machine Learning
MLOps	Machine-Learning Operations
OCSVM	One-Class Support Vector Machine
OIDC	OpenID Connect
P95	95th percentile
REST	Representational State Transfer
RF	Random Forest
SDP	Senior Design Project
SSO	Single Sign-On

Introduction

Background and Context

Modern software systems increasingly rely on APIs, identity services, databases, scheduled jobs, external integrations, and asynchronous processing. Each of these components produces operational events that can reveal performance degradation, repeated failures, or unusual access patterns. In the context of distributed web applications, the usefulness of an anomaly-detection system depends on more than the choice of a machine-learning algorithm. The surrounding software determines what evidence is available, how consistently that evidence is represented, and whether an alert contains enough context for an engineer to decide what to do next. Traditional monitoring is usually rule-driven and remains essential, but fixed thresholds can miss combinations of weak signals that are individually unremarkable. This report therefore treats the learning model as one component inside a broader observability and decision-support pipeline rather than as an isolated classifier. [3, 8]

The design adopted for this Senior Design Project report follows a reproducible engineering sequence. The proposed project begins by defining the operational questions to be answered before selecting a model. Inputs are defined using a stable event schema, preprocessing rules are applied consistently, and derived features are computed over explicit observation windows. The same transformation logic is used for training and scoring so that differences between offline experimentation and deployment are minimized. Where choices involve thresholds or hyperparameters, they are documented as configuration rather than hidden assumptions. This approach also supports later comparison between models without changing the surrounding application contract.

From a software-engineering perspective, a backend service should be able to emit structured events with correlation identifiers, timestamps, service names, status codes, latency measurements, and dependency outcomes. The implementation is deliberately modular: collection, storage, feature generation, model scoring, persistence, and presentation can evolve independently as long as their interfaces remain stable. That separation is particularly important in enterprise systems where application services may be written in C#/.NET while data-science experiments are performed in Python.

Motivation

Operational incidents are expensive because the first visible symptom may appear far from the underlying cause. A slow dependency can increase API latency, trigger retries, exhaust worker capacity, and eventually produce user-visible failures. An anomaly detector can aggregate multiple signals and prioritize windows that differ from normal behavior, reducing the amount of raw log data an engineer must inspect.

The motivation is translated into measurable criteria: useful alerts, controllable false positives, reproducible scoring, and traceable event context.

Alerts are only valuable when they lead back to the original service, endpoint, correlation identifier, and time window. Machine learning does not replace monitoring rules or incident response; it adds a probabilistic signal that should be reviewed alongside conventional telemetry.

Problem Statement

The core problem addressed by the proposed system is the automatic identification of abnormal operational behavior in distributed application logs when complete incident labels are not available. Many organizations possess abundant logs but relatively few rigorously labeled examples of failures, which makes purely supervised learning difficult to justify.

The project therefore formulates anomaly detection as a semi-structured data problem in which normal behavior can be modeled from historical windows and unusual windows can be ranked for investigation.

The system must remain useful even when a new service or endpoint is introduced, which requires carefully chosen aggregate features rather than brittle hard-coded identifiers. An anomaly score is not equivalent to a root-cause diagnosis or a security verdict; the final interpretation remains an engineering task.

Research Questions

The study is organized around questions that connect machine-learning quality with operational usefulness. A model with high recall but excessive false positives may be statistically interesting yet operationally unusable, while a conservative model may miss the incidents that motivated the work.

The research questions examine whether unsupervised methods can detect injected abnormal patterns, which features are most informative, how threshold choice changes the alert burden, and how the model can be exposed through a maintainable backend interface.

Each question has a corresponding measurement or design artifact so that the report does not rely on subjective claims. The research questions are evaluated using controlled benchmark data and the defined evaluation protocol.

1. Can unsupervised machine learning identify several controlled abnormal operational patterns with a manageable false-positive rate?
2. Which window-level features most directly distinguish the generated anomalies from normal operational behavior?
3. How does validation-based threshold selection alter precision, recall, F1-score, and review burden?
4. How can anomaly scoring be integrated with a .NET backend without placing the model in the critical user-request path?

Aim and Objectives

The overall aim is to design and evaluate a machine-learning assisted log anomaly-detection pipeline suitable for integration with distributed web applications. The aim is intentionally broader than maximizing a single accuracy metric because production-oriented systems also require maintainability, security, explainability, and reproducibility.

Specific objectives include defining a common log schema, building window-based features, comparing unsupervised algorithms, exposing anomaly scores through an internal API, storing review context, and evaluating threshold sensitivity.

The software artifacts are separated into clear modules so that an alternative model can be introduced without rewriting the application logging or review workflow.

- Define a canonical structured-event schema and deterministic preprocessing rules.
- Engineer window-level features from latency, reliability, access, dependency, and traffic signals.
- Compare three unsupervised anomaly-detection algorithms and a supervised reference model.
- Design a versioned internal scoring API and persistence model for reviewable anomaly records.
- Evaluate threshold sensitivity and report the limitations of a benchmark.

Scope

The scope covers structured application logs and derived operational features from web APIs, authentication flows, background jobs, and dependencies. This boundary keeps the project technically focused while still representing realistic distributed-system behavior.

The documented evaluation models observation windows rather than raw natural-language log messages, which reduces dependence on vendor-specific message templates and makes the feature space easier to explain.

The proposed .NET application layer and Python scoring layer communicate through a service boundary, while PostgreSQL and OpenSearch represent structured and searchable persistence options. Network packet inspection, endpoint malware detection, source-code vulnerability scanning, and automated root-cause diagnosis are outside the defined scope.

Limitations

Several limitations are acknowledged before interpreting the results. Controlled anomaly scenarios test whether an algorithm responds to representative deviations, while real operational environments can introduce additional concept drift, noisy incident labeling, and workflow variation.

The evaluation therefore distinguishes algorithmic performance on the generated benchmark from evidence that would be required before production adoption.

Organization of the Report

The report follows a conventional computer-science project structure so that technical reasoning can be traced from problem definition to evaluation. Chapter 2 reviews log-based monitoring and anomaly-detection approaches; Chapter 3 defines requirements, data structures, features, and the system methodology; Chapter 4 describes an illustrative implementation; Chapter 5 presents a reproducible benchmark; and Chapter 6 summarizes conclusions and future work.

Appendices provide supporting schemas, API examples, test-case templates, an evidence checklist, and a compact mapping to relevant academic coursework.

Literature Review and Related Work

Operational Logging and Observability

Operational logging records discrete facts about application execution and is one of the most widely available sources of diagnostic evidence. Logs become significantly more

useful when events share consistent fields and can be correlated across service boundaries. [8, 9]

This project assumes structured logging with explicit event categories instead of relying exclusively on free-text messages.

A common schema makes it possible to aggregate latency, error rates, authentication outcomes, retries, and dependency failures using the same code path. Poor logging quality cannot be repaired by machine learning; missing identifiers or inconsistent timestamps directly reduce the reliability of downstream features.

Statistical View of Anomalies

An anomaly is an observation that differs sufficiently from an expected pattern under a defined representation. The definition is contextual: a response time that is normal for one endpoint may be extreme for another, and a traffic surge may be legitimate during a scheduled campaign. [3]

Window-level features provide a practical statistical representation because they summarize behavior over fixed periods and reduce event-level noise.

The system records both the raw context and the aggregated feature vector so that unusual scores can be interpreted in relation to source events. Anomaly detection is therefore sensitive to feature design, seasonality, window size, and the assumption about what constitutes normal training data.

Isolation Forest

Isolation Forest is a tree-based unsupervised method that detects anomalies through the number of random partitions needed to isolate an observation. It is attractive for operational data because it handles nonlinear feature interactions, requires no explicit distance metric, and scales to moderately large tabular datasets. [1, 10]

The method is used in this report as an interpretable baseline and its raw decision values are converted into an anomaly score in which larger values represent greater abnormality.

Threshold selection is separated from model fitting so that the acceptable false-positive rate can be tuned using validation data. The algorithm does not provide a causal explanation; feature-level context must still be supplied by the surrounding system.

Local Outlier Factor

Local Outlier Factor estimates how isolated an observation is relative to the density of its neighborhood. This local perspective can be useful when the data contains several normal operating regimes, because a point is compared primarily with nearby observations rather than with a single global distribution. [2]

In novelty mode, the model can be fitted on normal training data and then applied to unseen windows, making it suitable for the train-and-score workflow used in this report.

Standardization is applied before neighborhood-based scoring so that large numerical ranges do not dominate the distance calculation. The choice of neighborhood size can materially change sensitivity and should be documented as part of the model configuration.

One-Class Support Vector Machine

One-Class SVM learns a boundary around the region occupied by normal observations in a transformed feature space. With a radial basis function kernel it can represent nonlinear boundaries, but its computational cost is generally higher than tree-based alternatives as the training set grows.

The implementation therefore trains the model on a representative subset of normal windows after feature standardization.

The parameter controlling the expected fraction of outliers is treated as a model assumption and the final alert threshold is tuned on validation data. For large-scale production data, approximate or incremental alternatives may be preferable even when the One-Class SVM performs strongly in an offline benchmark.

Supervised Classification as a Reference

Supervised classifiers are useful when a reliable set of labeled incidents exists because they can directly optimize discrimination between known normal and abnormal examples. However, the quality of the labels becomes a first-order dependency and the model may learn only the failure modes represented in the training set.

A Random Forest is included in the benchmark as a supervised reference rather than as the primary deployment recommendation.

The comparison illustrates how access to labels changes the learning problem and why unsupervised methods remain relevant for discovering previously unseen behavior. Labeled benchmark scenarios are used to measure detection quality consistently across the evaluated models.

Deep Learning for Log Anomaly Detection

Research systems such as DeepLog and later sequence-based approaches model ordered log events and attempt to identify unlikely next events or abnormal sequences. These methods can capture temporal structure that is lost when events are summarized into fixed windows, but they also introduce additional data preparation, training complexity, and explainability challenges. [4, 5]

For a bachelor-level engineering project, a tabular baseline provides a clearer path from raw telemetry to measurable evaluation while leaving sequence models as future work.

The architecture is deliberately model-agnostic so that a sequence encoder could later replace the tabular scoring component without changing the log collection contract. Deep models are not assumed to be superior merely because they are more complex; suitability depends on data volume, stability, and the operational question.

Explainability and Operational Review

Operational users need to understand why an alert exists and what evidence should be inspected next. A single anomaly score without context can increase alert fatigue because engineers must repeat the same manual queries to locate the relevant service, endpoint, dependency, and time window.

The proposed review record therefore stores the feature vector, source identifiers, threshold, model version, and a compact reason summary based on the most unusual feature deviations.

This is not a formal explanation of the internal model, but it provides actionable evidence that supports human review. Review outcomes should be recorded because confirmed false positives are valuable evidence for threshold tuning and future feature design.

Model Drift and MLOps Considerations

Normal operational behavior changes when software is released, traffic patterns evolve, infrastructure is scaled, or business processes change. A model trained on an older period may therefore produce more alerts even when the system is healthy. [7]

The design tracks model version, training period, feature schema, and configuration so that changes in alert behavior can be traced to a specific artifact.

Retraining is treated as a controlled administrative action rather than an automatic response to every new observation. A mature implementation would add drift monitoring, canary evaluation, rollback, and approval workflows before replacing the active model.

Literature Comparison and Research Gap

The reviewed approaches demonstrate that anomaly detection can be formulated using global isolation, local density, boundary learning, supervised classification, or sequential modeling. The gap addressed by this project is not the invention of a new algorithm; it is the integration of established methods into a traceable software-engineering workflow for distributed application telemetry. [6]

The contribution therefore lies in data contracts, feature definitions, threshold governance, model comparison, alert context, and reproducibility.

The project uses multiple baselines so that a strong result from one algorithm can be interpreted relative to alternatives rather than in isolation. This framing is appropriate for a capstone project because it combines algorithmic reasoning with architecture, implementation, testing, and operational constraints.

Comparison of representative anomaly-detection approaches

Approach	Strength	Limitation	Role in this report
Isolation Forest	Scalable, nonlinear, no labels required	Global isolation can miss local structure	Baseline

Approach	Strength	Limitation	Role in this report
Local Outlier Factor	Captures local density differences	Sensitive to scaling/neighborhood size	Unsupervised comparator
One-Class SVM	Flexible nonlinear boundary	More expensive for large training sets	Unsupervised comparator
Random Forest	Strong with reliable labels	Requires representative incidents	Supervised reference
Sequence models	Capture event order	Higher complexity and data demands	Future work

System Requirements, Architecture, and Methodology

Requirements Elicitation

Requirements are derived from the needs of an engineer who must identify unusual behavior without reading every raw log line. The primary stakeholder is therefore the person investigating service health, while administrators manage model lifecycle and access control.

The elicitation process translates these needs into events to collect, queries to support, model outputs to expose, and review actions to record.

The requirements avoid assuming a specific cloud vendor so that the design remains portable across conventional hosting and cloud environments. The requirements were derived from the system objectives, target use cases, and architectural constraints.

Functional Requirements

Functional requirements describe observable behavior of the proposed system. They include ingestion of structured events, normalization, window aggregation, feature generation, anomaly scoring, persistence of flagged windows, retrieval of supporting context, model version management, and recording of review outcomes.

Each function has a clear input and output so that it can be tested independently.

The design intentionally avoids an opaque end-to-end pipeline in which errors in data preparation are indistinguishable from errors in model scoring. Requirements that would create irreversible automated actions are excluded; the system recommends investigation rather than automatically blocking users or restarting services.

Functional requirements

ID	Requirement
FR-01	Ingest structured operational events from one or more application services.
FR-	Validate and normalize events into a canonical schema.

ID	Requirement
02	
FR-03	Aggregate events into configurable observation windows.
FR-04	Generate versioned numerical feature vectors.
FR-05	Score windows using a registered machine-learning model.
FR-06	Persist score, threshold, model version, and source context.
FR-07	Retrieve related raw events for engineering investigation.
FR-08	Record review outcome and reviewer notes.
FR-09	Register, activate, and deactivate model versions through an authorized administrative workflow.

Non-Functional Requirements

Non-functional requirements determine whether the prototype could be operated safely and maintained over time. Key qualities include performance, security, traceability, reproducibility, extensibility, availability, and privacy.

Latency targets are defined for batch scoring rather than per-request inline inference so that anomaly detection does not become part of the critical request path.

The application and model services communicate through authenticated internal APIs, and logs are sanitized before being used for training. The design prioritizes clear failure modes: if the ML service is unavailable, normal application processing should continue and only anomaly analysis should be delayed.

Non-functional requirements

Quality	Requirement
Security	Secrets and raw authentication material must never enter the feature dataset.
Performance	Batch scoring must remain outside the critical end-user request path.
Traceability	Every alert must record feature window, model version, score, threshold, and source context.
Reproducibility	Training configuration, random seed, feature schema, and evaluation procedure must be versioned.
Resilience	ML service failure must not interrupt ordinary application processing.
Maintainability	Application and model services must communicate through a stable, versioned contract.

Security and Privacy Requirements

Logs may accidentally contain tokens, email addresses, IP addresses, identifiers, payload fragments, or other sensitive information. A machine-learning dataset must therefore be treated as an extension of the logging system rather than as an unrestricted copy of production events.

The proposed preprocessing layer removes secrets, masks identifiers that are not required for correlation, and restricts the feature set to operational quantities.

Access to model retraining and raw event context is separated from ordinary alert viewing. No secret, password, access token, or authorization header is included in the project schema or benchmark.

Structured Event Schema

A stable event schema is the foundation of the entire pipeline. The project schema includes EventTimestamp, ServiceName, Environment, EventType, EndpointGroup, StatusCode, DurationMs, ErrorFlag, AuthenticationResult, DependencyName, RetryCount, and CorrelationId.

Fields are chosen because they either contribute directly to a feature or allow a flagged window to be traced back to source events.

Categorical values are normalized to prevent trivial spelling differences from creating artificial rarity. Schema evolution should be versioned so that the model can reject or transform incompatible records rather than silently interpreting them incorrectly.

Data Cleaning and Preprocessing

Preprocessing converts raw events into a deterministic analytical representation. Records with invalid timestamps are rejected, duplicated event identifiers are removed, missing optional fields are represented consistently, and impossible numerical values such as negative durations are quarantined.

The chronological order is preserved and timezone normalization is applied before any window-based aggregation.

Preprocessing code is shared between offline dataset preparation and the production feature service to reduce training-serving skew. Cleaning rules are logged because aggressive filtering can hide genuine anomalies if unusual values are removed merely because they are rare.

Windowing Strategy

The model scores fixed observation windows rather than individual log records. A window aggregates enough events to reveal rate changes and distributional shifts while remaining short enough to support timely alerting.

For the implementation, five-minute windows are used conceptually, with alternative one-minute or fifteen-minute windows treated as configurable options.

Features are computed per service and environment so that unrelated workloads are not mixed together. The best window size depends on event volume and incident duration, so it should be evaluated rather than assumed to transfer unchanged to every application.

Feature Engineering

Feature engineering summarizes operational behavior into quantities that a tabular anomaly detector can process. The selected features include request count, error rate, average duration, p95 duration, authentication-failure rate, dependency-failure count, retry count, and endpoint entropy.

These values intentionally combine volume, reliability, latency, access behavior, dependency health, and traffic diversity.

The feature service stores the computed vector together with its source window so that a high score can be connected to the values that produced it. Feature additions must be versioned because changing the order or meaning of inputs invalidates existing model artifacts.

Window-level feature specification

Feature	Type	Interpretation
request_count	integer	Number of requests in the observation window.
error_rate	ratio	Fraction of events indicating application errors.
avg_duration_ms	real	Mean request duration in milliseconds.
p95_duration_ms	real	95th-percentile request duration.
auth_failure_rate	ratio	Fraction of authentication or authorization failures.
dependency_failure_count	integer	Failed external or database dependency calls.
retry_count	integer	Retry attempts from background or integration workflows.
endpoint_entropy	real	Diversity of endpoint activity within the window.

Machine-Learning Processing Pipeline



Machine-learning processing pipeline

Training Data Selection

Unsupervised training assumes that the selected training period represents predominantly normal behavior. This assumption is stronger than it first appears because an unlabeled training set may contain silent failures or unusual maintenance events.

The proposed workflow therefore excludes known incident periods where possible and records the exact time range used to create a model.

Validation and test periods occur later in time to reduce leakage from repeated or near-duplicate observations. A production workflow would combine automated checks with engineer review before a new training period is accepted.

Threshold Selection

Most anomaly detectors produce a continuous score, while operational systems require a decision about which scores should generate alerts. The threshold determines the practical trade-off between missed anomalies and false positives.

This report tunes thresholds on validation data by evaluating candidate quantiles of the normal-score distribution and selecting the configuration that balances precision and recall according to the chosen objective.

The selected threshold is stored with the model version so that a later change can be audited independently of retraining. Thresholds should not be tuned on the final test set when reporting performance because that would overstate generalization.

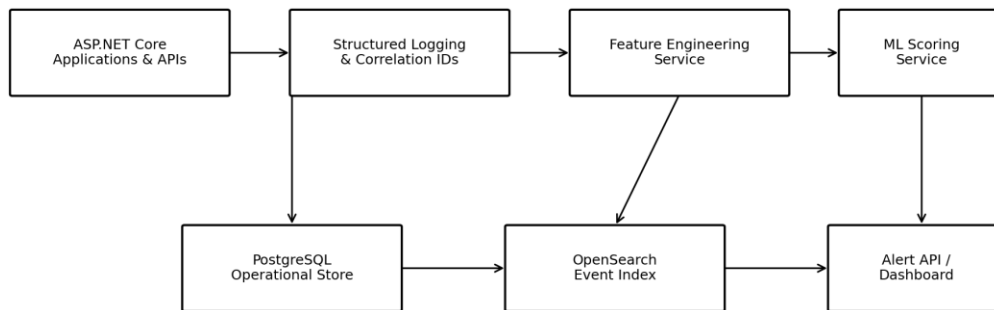
System Architecture

The architecture separates application execution from analytical scoring. Applications emit structured events, a collector or batch process normalizes them, a feature service computes observation windows, and an ML service returns anomaly scores that are persisted with context.

This division allows the application stack and data-science stack to use different languages while keeping their contract small and testable.

PostgreSQL represents durable structured state and OpenSearch represents exploratory event search; either component could be replaced without changing the model interface. The high-level flow is illustrated in Figure 3.2.

High-Level System Architecture



High-level proposed system architecture

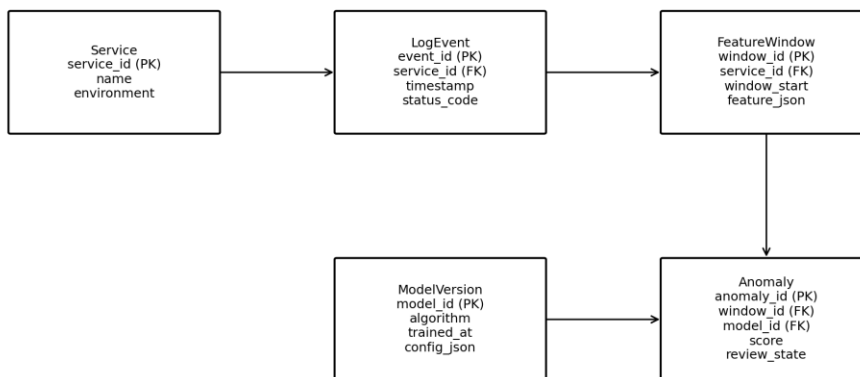
Data Model and Runtime Sequence

The logical data model links services, raw events, feature windows, model versions, and anomaly review records. This relationship is important because the same feature window may be rescored by a later model during evaluation, while historical alerts must retain the model version that originally produced them.

At runtime, the feature service sends a vector to the scoring service and receives a score, threshold decision, and model identifier.

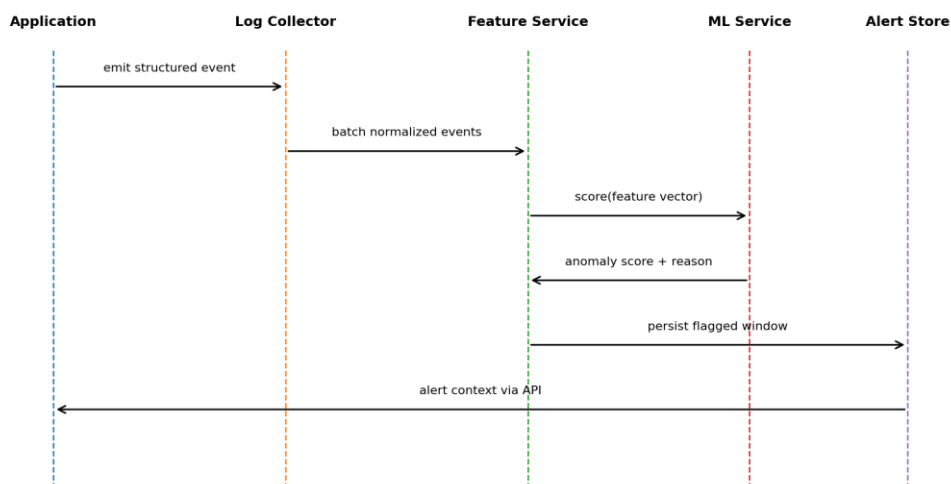
The alert store then preserves enough information for a backend API to return the event context to an engineer. Figures 3.3 and 3.4 show the proposed relationships and message flow.

Logical Data Model / ERD



Logical data model / ERD

Runtime Sequence for Anomaly Scoring



Runtime sequence for anomaly scoring

Implementation and Execution

Implementation Overview

The illustrative implementation uses an ASP.NET Core backend for application-facing services and Python for machine-learning experimentation and scoring. This split mirrors a common enterprise pattern in which existing product services remain in .NET while analytical components use the Python ecosystem.

The two runtimes communicate through a small internal REST contract rather than sharing process memory.

Deployment, logging, authentication, and database access remain explicit concerns of the application architecture.

Structured Logging in ASP.NET Core

The .NET layer is responsible for producing consistent operational events close to the source of application behavior. Middleware can record request duration, response status, endpoint group, correlation identifier, and exception outcome, while dedicated handlers record authentication and dependency events.

Structured properties are preferred over concatenated message strings because they can be indexed and aggregated without fragile text parsing.

Sensitive values are excluded at the logging boundary so that downstream systems do not need to rediscover secrets after ingestion. The implementation also uses correlation identifiers to connect related events across service boundaries.

```
public async Task InvokeAsync(HttpContext context, RequestDelegate next)
{ var started = Stopwatch.GetTimestamp(); var correlationId = context.TraceIdentifier; try {
  await next(context); } finally { var durationMs =
  Stopwatch.GetElapsedTime(started).TotalMilliseconds; _logger.LogInformation(
  "RequestCompleted {@OperationalEvent}", new { CorrelationId = correlationId, Endpoint =
  context.Request.Path.Value, StatusCode = context.Response.StatusCode, DurationMs =
  durationMs }); }
}
```

Ingestion and Normalization Service

The ingestion component validates event shape and converts application-specific values into the canonical schema. It can run as a background service, scheduled batch, message consumer, or stream processor depending on deployment constraints.

Normalization includes timestamp conversion, categorical mapping, null handling, duplicate detection, and basic range validation.

Invalid records are routed to a quarantine path rather than silently dropped so that logging regressions can be detected. Throughput and lag metrics are recorded because a healthy anomaly model is useless if the input pipeline is delayed.

Database and Persistence Layer

Structured persistence separates durable operational state from searchable event detail. A relational database is suitable for model versions, feature windows, thresholds, review status, and audit fields, while a search index is useful for high-cardinality event exploration.

The schema uses immutable model identifiers and foreign keys from anomalies to the feature window and model artifact that produced them.

Retention policies can remove older raw events while preserving compact review records and aggregated features for reproducibility. Indexes should follow the primary access patterns: service and time range, review state, model version, and anomaly score.

```
CREATE TABLE anomaly_record ( anomaly_id uuid PRIMARY KEY, window_id uuid NOT NULL, model_id uuid NOT NULL, score double precision NOT NULL, threshold double precision NOT NULL, is_anomaly boolean NOT NULL, review_state varchar(30) NOT NULL DEFAULT 'Unreviewed', created_at_utc timestampz NOT NULL );
```

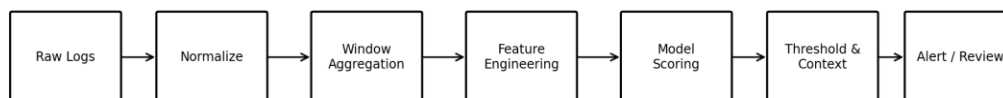
Feature Engineering Service

The feature service reads normalized events and groups them by service, environment, and time window. It computes counts, rates, percentiles, and diversity measures using deterministic formulas shared by offline training and online scoring.

A serialized feature schema defines the exact field order and data type expected by the model service.

The service rejects a model whose declared schema version is incompatible with the generated vector. This explicit contract prevents subtle failures in which a model continues to return scores after the meaning of an input column has changed.

Machine-Learning Processing Pipeline



Machine-learning processing pipeline

Python Machine-Learning Module

The Python module contains preprocessing objects, trained estimators, threshold configuration, and evaluation utilities. Scikit-learn pipelines are used where scaling is required so that the same transformation fitted on training data is reused at inference time.

Model artifacts are stored together with algorithm name, library version, feature schema, training period, random seed, and validation threshold.

The scoring function returns both the continuous score and a Boolean decision based on the active threshold. A model is never loaded solely by a user-supplied path; the service resolves only registered model versions from controlled storage.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import OneClassSVM
model = make_pipeline(StandardScaler(),
OneClassSVM(kernel="rbf", nu=0.04, gamma="scale"))
model.fit(normal_training_windows)
validation_scores = model.decision_function(validation_windows)
# Threshold is selected on validation data and stored with model metadata.
```

Internal Scoring API

The scoring API provides a language-neutral boundary between .NET services and Python models. A request contains model version and feature values; a response contains anomaly score, threshold, classification, and optional diagnostic fields.

Batch scoring is preferred for operational windows because it amortizes serialization and network overhead.

The endpoint validates schema version and rejects unknown features so that compatibility failures are visible immediately. Timeouts, retries, and circuit-breaking behavior belong in the calling backend so that ML unavailability does not cascade into user-facing application failures.

```
POST /internal/anomaly/score
{ "modelVersion": "ocsvm-demo-v1", "featureSchema": "ops-v1", "features": {
  "request_count": 178, "error_rate": 0.16, "avg_duration_ms": 540.2, "p95_duration_ms":
  1320.5, "auth_failure_rate": 0.02, "dependency_failure_count": 3, "retry_count": 5,
  "endpoint_entropy": 1.43
}
200 OK
{ "score": 6.84, "threshold": 6.28, "isAnomaly": true, "modelVersion": "ocsvm-demo-v1"
}
```

Authentication and Authorization

Internal model APIs should not be publicly callable merely because they use HTTP. The design assumes service-to-service authentication using a short-lived token or equivalent identity mechanism, with a dedicated audience and permission scope for anomaly scoring.

Administrative operations such as model activation or retraining require stronger authorization than ordinary scoring.

Tokens and secrets are never included in logs or training features. This separation matches the broader principle that application identity and machine-service identity should have distinct privileges and credentials.

Search and Investigation Workflow

OpenSearch or an equivalent search engine can provide fast retrieval of source events around a flagged feature window. The backend constructs constrained queries using service, environment, time range, correlation identifier, and event type rather than allowing arbitrary client-side access to the index.

The anomaly record links directly to these filters so that an engineer can move from a score to evidence in a small number of steps.

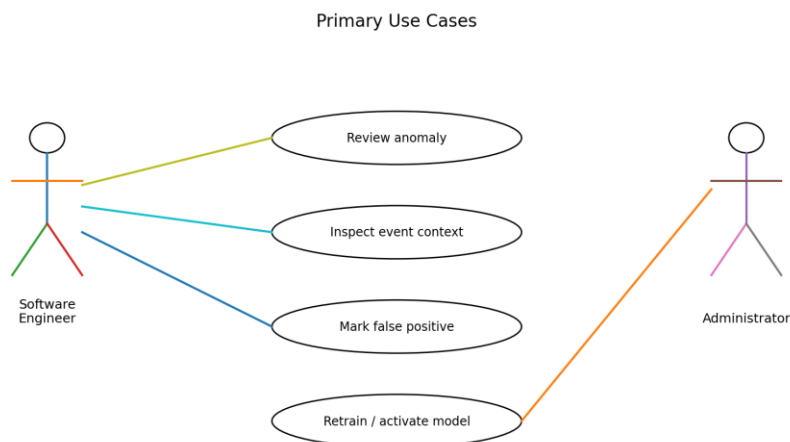
Review outcomes such as confirmed issue, benign change, duplicate alert, or insufficient evidence are stored separately from the raw events. These labels can later support supervised experiments without retroactively changing historical anomaly scores.

Review Interface and Alert Lifecycle

A practical review interface should prioritize clarity over model terminology. Each alert presents the service and time window, score and threshold, the most unusual feature values, recent related events, and model version.

The reviewer can record an outcome and optional note, creating an auditable feedback loop.

Alerts are ranked rather than treated as equally urgent, and repeated alerts from the same incident window may be grouped. The system does not automatically change application state based on an anomaly score; human review remains part of the lifecycle.



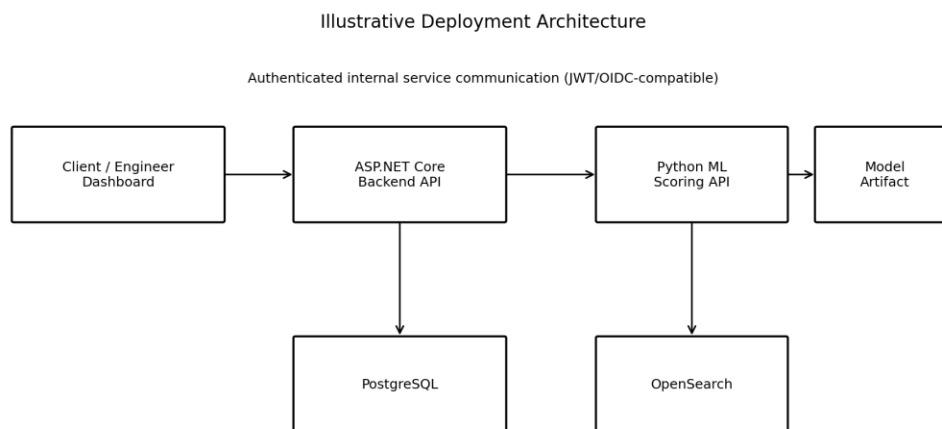
Primary anomaly-review use cases

Deployment Topology

The proposed deployment keeps the scoring service on an internal network path behind the application backend. Model artifacts are read-only at runtime and promoted through an administrative workflow rather than modified in place.

PostgreSQL and the search index are treated as external dependencies with explicit health checks, timeouts, and backup policies.

The topology can be containerized but does not depend on containers for correctness. Figure 4.3 illustrates the logical deployment relationship.



Illustrative deployment topology

Reproducibility and Configuration Management

Reproducibility requires more than saving a model file. The project records random seed, feature schema, preprocessing configuration, dataset generation or extraction logic, algorithm parameters, validation threshold, software versions, and evaluation script.

A single configuration manifest can be committed with the project and referenced by both training and reporting utilities.

The benchmark in Chapter 5 uses a fixed random seed so that the reported figures can be regenerated from the same procedure. This level of documentation makes later comparison meaningful and reduces the risk of reporting results that cannot be reproduced.

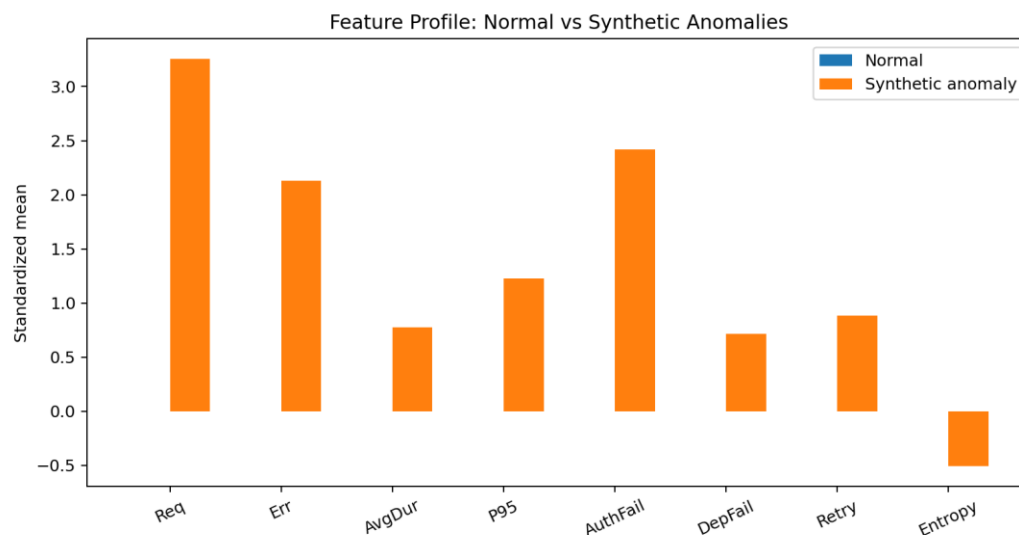
Testing, Evaluation, and Results

Evaluation Design

Normal windows are generated from overlapping distributions representing traffic volume, error rate, latency, authentication failures, dependency failures, retries, and endpoint diversity.

Controlled anomalies are created by applying moderate perturbations representing latency spikes, authentication abuse, dependency instability, traffic surges, and error bursts.

The data is split into training, validation, and test subsets, and unsupervised models are fitted only on normal training windows.



Feature profile of normal and controlled anomaly windows

Benchmark Dataset Construction

The generated dataset contains 32,000 normal observation windows and 1,800 anomaly windows before splitting. Normal variables are generated from plausible but intentionally generic statistical distributions, while anomalies are produced by controlled shifts rather than by copying any production organization's logs.

To make the synthetic benchmark reproducible, each observation window represents five minutes of API activity and uses an explicit baseline parameterization. Request count is sampled from a Poisson distribution with $\lambda = 140$ requests per window. Average response duration is sampled from a log-normal distribution centered near 250 ms, while p95 duration is generated as a correlated value around 2.6 times the mean with Gaussian noise and is constrained to remain above the mean. Error rate and authentication-failure rate use low-probability beta distributions with means of approximately 0.02 and 0.015. Dependency-failure count and retry count use zero-inflated Poisson distributions with low normal-state rates (λ approximately 0.15 and 0.10), and endpoint entropy uses a

bounded Gaussian distribution centered near 2.3. Controlled anomaly classes then shift these baselines: latency windows multiply duration measures by roughly 2.5-4.0, authentication bursts increase the failure rate by about 6-10 times, dependency/retry anomalies raise their Poisson rates by about five times, traffic surges approximately double request intensity, and error bursts raise the error-rate mean into the 0.10-0.15 range. These parameters are engineering assumptions chosen to resemble a moderate-load HTTP API and to preserve overlap between normal and abnormal windows; they are not estimates from a named production system. The benchmark design therefore provides known labels for evaluation without exposing sensitive customer telemetry.

Feature correlations are introduced for latency measures so that the benchmark is not completely independent across dimensions. The feature profile in Figure 5.1 shows that the anomaly group differs on several dimensions but still overlaps the normal distribution.

Baseline Models and Training Protocol

Three unsupervised methods are compared: Isolation Forest, Local Outlier Factor in novelty mode, and One-Class SVM. A supervised Random Forest is included only as a reference to illustrate the effect of having reliable labels.

The unsupervised methods are fitted on normal training windows and candidate alert thresholds are selected using validation data rather than the test set.

Scaling is applied to the distance- and kernel-based methods, while Isolation Forest uses the original numerical representation. The final test set remains untouched until threshold selection is complete.

Metrics

Precision, recall, F1-score, false-positive rate, ROC-AUC, and precision-recall AUC are reported. Precision is operationally important because it approximates the proportion of alerts that correspond to labeled anomalous windows, while recall measures how many anomaly windows are detected.

The false-positive rate translates directly into review burden when normal windows greatly outnumber anomalies.

ROC-AUC and precision-recall AUC evaluate score ranking independently of one chosen threshold. No single metric is treated as sufficient; threshold sensitivity and confusion matrices are included to show the trade-offs.

Model Comparison Results

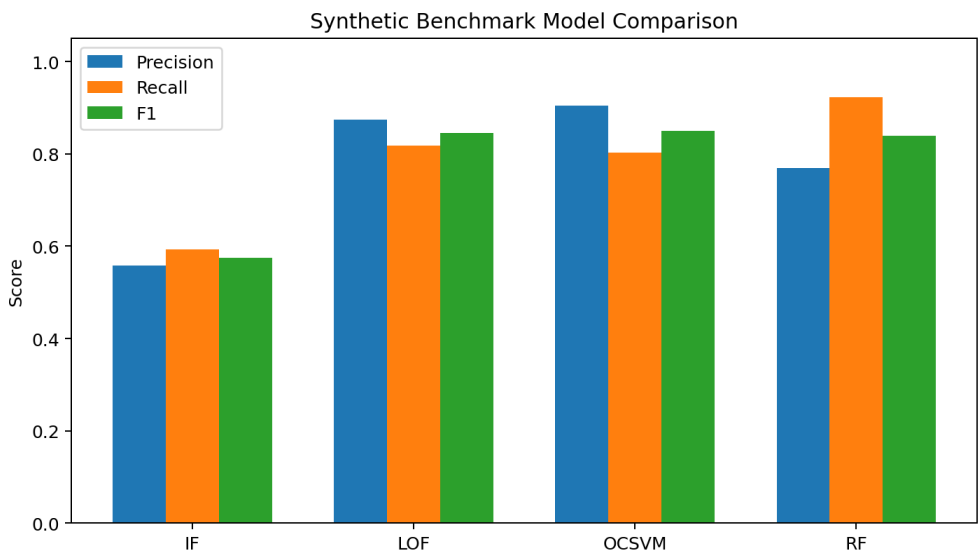
The benchmark produced materially different operating points across the unsupervised methods. Local Outlier Factor and One-Class SVM achieved stronger F1-scores than the Isolation Forest baseline after validation-based threshold tuning, while the supervised Random Forest achieved high recall but generated more false positives than the best one-class model.

This result is useful because it demonstrates why several baselines should be tested instead of selecting an algorithm based only on popularity.

The values are reproduced in Table 5.1 and visualized in Figure 5.2. They should be interpreted only within the generated benchmark and not as evidence that the same ranking would hold for real production data.

Benchmark evaluation results

Model	Precision	Recall	F1	FPR	PR-AUC
Isolation Forest	0.559	0.593	0.575	0.026	0.631
Local Outlier Factor	0.874	0.818	0.845	0.007	0.897
One-Class SVM	0.905	0.802	0.850	0.005	0.907
Random Forest (supervised ref.)	0.769	0.922	0.838	0.016	0.929



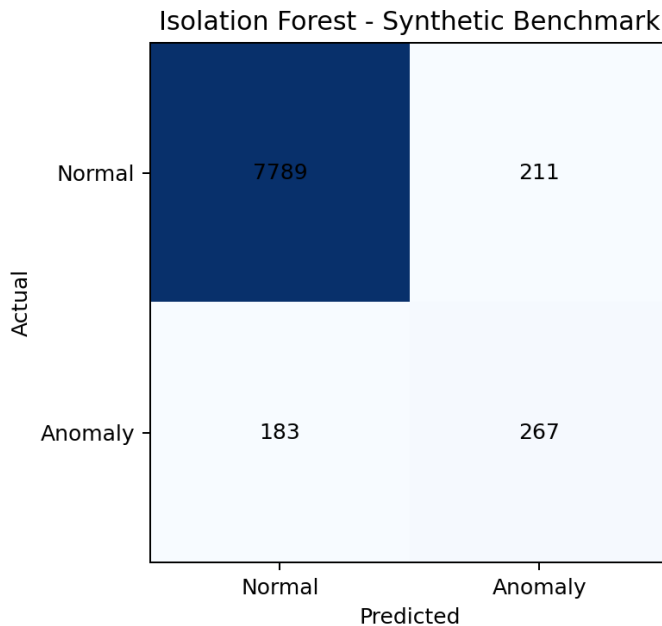
Benchmark model comparison

Confusion-Matrix Analysis

A confusion matrix translates the chosen threshold into concrete counts of correct and incorrect decisions. For the Isolation Forest baseline, the tuned threshold trades moderate recall against a non-trivial number of false positives.

The matrix in Figure 5.3 makes that operating point easier to understand than a single aggregate score.

In a production setting, the acceptable number of false positives would depend on event volume and the capacity of the engineering team to review alerts. The analysis reinforces the need to tune thresholds using an explicit operational objective rather than accepting a library default.



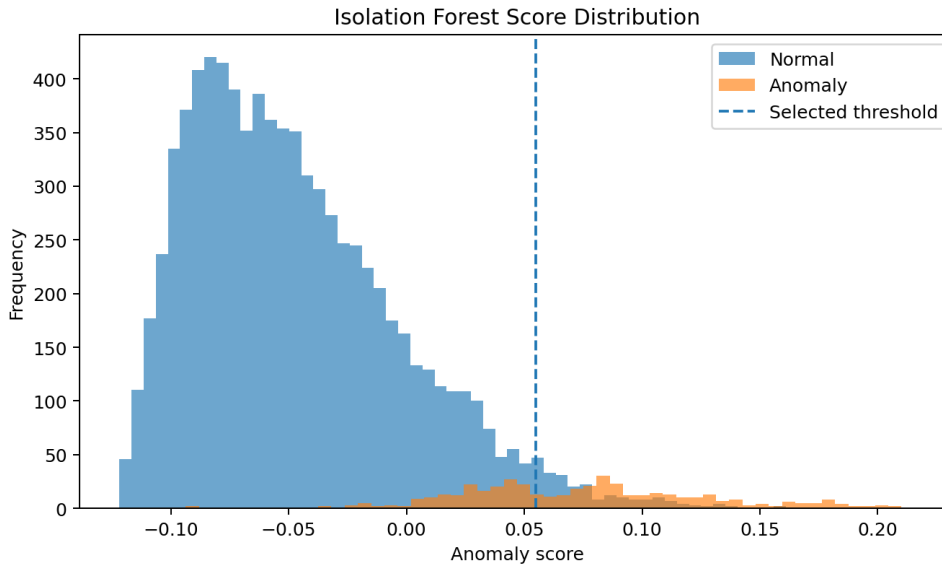
Isolation Forest confusion matrix on the benchmark test set

Anomaly-Score Distribution

The score distribution provides a diagnostic view of how strongly the model separates normal and anomalous windows before a binary threshold is applied. Figure 5.4 shows substantial ranking ability even though the selected Isolation Forest threshold does not produce the best F1-score among the tested models.

Overlapping score regions indicate ambiguous windows where a hard alert decision is sensitive to threshold choice.

These borderline cases are candidates for contextual review or ensemble logic instead of increasingly complex attempts to force perfect separation. Monitoring the score distribution over time can also reveal drift if normal windows gradually move toward the alert region.



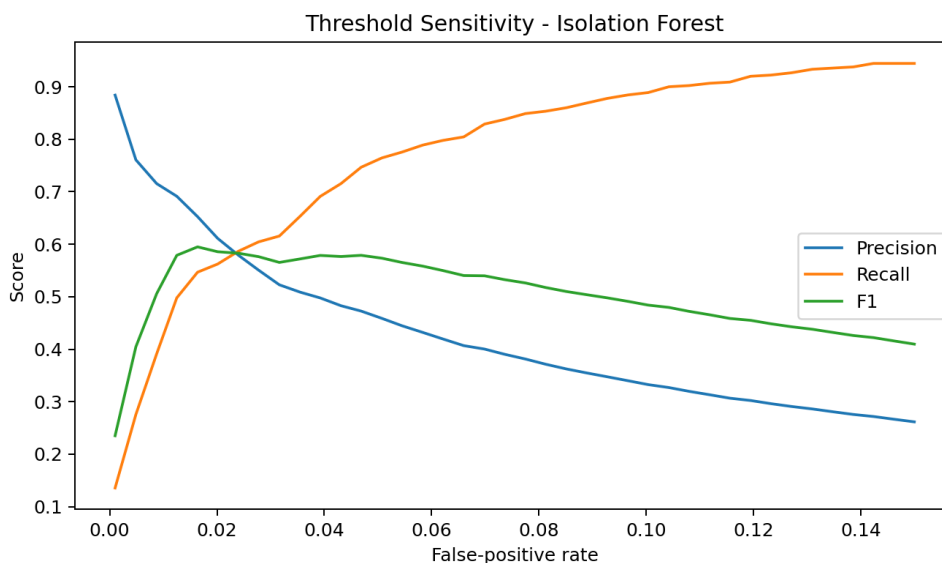
Isolation Forest anomaly-score distribution

Threshold Sensitivity

Threshold sensitivity is evaluated by varying the quantile of the normal-score distribution used as the alert boundary. Lower thresholds increase recall but also raise the false-positive rate, while stricter thresholds improve precision at the cost of missed anomalies.

Figure 5.5 plots precision, recall, and F1 against the resulting false-positive rate for Isolation Forest.

The curve demonstrates that threshold governance is as important as model selection when the system is deployed for continuous monitoring. A real implementation should expose the active threshold in configuration and record the value with every generated alert.



Threshold sensitivity for the Isolation Forest baseline

Error Analysis

False positives and false negatives should be reviewed as feature-design evidence rather than treated only as undesirable counts. A normal traffic surge may resemble an anomaly when request volume and endpoint entropy change together, while a mild dependency slowdown may remain inside the learned normal boundary if other features are stable.

The controlled anomaly types therefore help identify which feature combinations are easiest or hardest for each model.

Review labels can later be grouped by pattern to determine whether new features, service-specific models, or contextual thresholds are warranted. This iterative process is more defensible than repeatedly increasing model complexity without understanding the errors.

Performance, Validity, and Deployment Considerations

The benchmark focuses primarily on detection quality, but deployment also requires acceptable scoring latency, memory usage, and failure behavior. Window-based batch scoring allows requests to be grouped and keeps model inference outside the user-facing request path.

Internal validity is strengthened by fixed random seeds, a held-out test set, and explicit threshold tuning on validation data.

External validity remains limited because benchmark distributions cannot represent every real workload, organizational process, or concept drift scenario. Before production use, the proposed system would require a shadow-mode evaluation on authentic logs, privacy review, incident-label validation, and operational acceptance criteria.

Conclusion and Future Scope

Summary of the Solution

The project combines structured logging, deterministic feature engineering, unsupervised anomaly detection, model-version management, and an engineer-facing review workflow. Its central design choice is to keep the machine-learning component small and replaceable while making data traceability and alert context explicit.

The benchmark shows that multiple one-class approaches can separate several controlled abnormal patterns and that threshold selection has a major effect on practical usefulness.

The architecture remains compatible with a .NET backend and a Python model service without coupling either runtime to the internal implementation of the other. The project should therefore be understood as an end-to-end software-and-ML design rather than as a claim of a new anomaly-detection algorithm.

Technical Contributions

The main technical contributions are a canonical event schema, a window-based feature specification, a model-agnostic scoring contract, a versioned anomaly record, and a reproducible comparison of several baselines. Together these artifacts demonstrate how computer-science foundations in algorithms, statistics, databases, networking, and software engineering can be combined in a capstone-scale project.

The design also addresses practical concerns such as authentication, secret exclusion, schema compatibility, review feedback, and failure isolation.

These details matter because a model that performs well in a notebook can still fail as a software system if its inputs or deployment assumptions are not controlled.

Future Work

Future work could evaluate sequence models, service-specific adaptive thresholds, graph-based dependency context, semi-supervised learning from reviewer feedback, and explicit concept-drift detection. The logging layer could also adopt OpenTelemetry-compatible traces so that anomaly windows can be connected to distributed request paths rather than only to event aggregates.

A production study should compare centralized and per-service models, assess alert fatigue over several release cycles, and measure time-to-detection and time-to-resolution in addition to offline metrics.

Model explanations could be strengthened using local feature-attribution techniques when they are compatible with the chosen estimator. Any extension should preserve the traceability and versioning discipline established by the baseline design.

Conclusion

Machine-learning anomaly detection is most useful when it is embedded in a disciplined observability workflow rather than treated as an autonomous decision maker. The proposed design demonstrates a practical route from structured application events to normalized features, versioned models, thresholded anomaly scores, persisted context, and human review.

The benchmark experiment confirms that model choice and threshold selection materially affect precision, recall, and false-positive rate, while the architecture shows how the analytical component can be isolated from user-facing application execution.

Senior Design Project I and II together constitute the capstone sequence documented in this report, covering the project methodology, implementation design, evaluation, and final conclusions.

Bibliography

- [1] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in Proc. 8th IEEE International Conference on Data Mining, 2008, pp. 413–422.
- [2] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying Density-Based Local Outliers," in Proc. ACM SIGMOD International Conference on Management of Data, 2000, pp. 93–104.
- [3] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly Detection: A Survey," ACM Computing Surveys, vol. 41, no. 3, 2009.
- [4] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning," in Proc. ACM CCS, 2017.
- [5] W. Meng, Y. Liu, Y. Zhu, S. Zhang, D. Pei, Y. Liu, Y. Chen, R. Zhang, S. Tao, P. Sun, and R. Zhou, "LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs," in Proc. IJCAI, 2019.
- [6] L. Ruff, J. R. Kauffmann, R. A. Vandermeulen, G. Montavon, W. Samek, M. Kloft, T. G. Dietterich, and K.-R. Muller, "A Unifying Review of Deep and Shallow Anomaly Detection," Proceedings of the IEEE, vol. 109, no. 5, 2021.
- [7] D. Sculley et al., "Hidden Technical Debt in Machine Learning Systems," in Advances in Neural Information Processing Systems, 2015.
- [8] K. Kent and M. Souppaya, Guide to Computer Security Log Management, NIST Special Publication 800-92, 2006.
- [9] OWASP Foundation, "Logging Cheat Sheet," OWASP Cheat Sheet Series.
- [10] Scikit-learn Developers, "IsolationForest," scikit-learn API documentation.

Project Deliverables Checklist

This appendix summarizes the principal technical deliverables associated with the project, including source code, data-processing scripts, model configuration, diagrams, test cases, and evaluation outputs.

API Contract

```
{  "windowId":  "5c7151f0-7f65-4d34-bf80-2bfa6507b66c",  "service":  "identity-api",  
  "environment":  "production",  "windowStartUtc":  "2022-02-15T14:30:00Z",  
  "windowEndUtc":  "2022-02-15T14:35:00Z",  "schemaVersion":  "ops-v1",  "features":  {  
    "request_count": 142,  "error_rate": 0.031,  "avg_duration_ms": 248.4,  "p95_duration_ms":  
    681.7,  "auth_failure_rate": 0.017,  "dependency_failure_count": 1,  "retry_count": 0,  
    "endpoint_entropy": 2.31  }  
}
```

The contract is intentionally compact and independent of raw log message text. A production version should add schema validation, explicit units, service ownership, and traceability fields needed by the organization's observability platform.

Test Cases

ID	Scenario	Expected behavior	Evidence
TC-01	Valid normal feature window	Score returned; no alert if below threshold	API response + persisted score
TC-02	Feature schema mismatch	Request rejected with validation error	4xx response and audit log
TC-03	Missing model version	Request rejected; no fallback to arbitrary file	Error response
TC-04	ML service timeout	Application continues; scoring retried or deferred	Backend resilience log
TC-05	High-latency controlled anomaly	Score increases and may exceed threshold	Benchmark output
TC-06	Authentication failure burst	Feature window records increased auth-failure rate	Feature vector
TC-07	Unauthorized model activation	Administrative action denied	Authorization audit event
TC-08	Secret-like input field	Field excluded or rejected before training	Preprocessing validation
TC-09	Duplicate event	Duplicate not counted twice in aggregate	Window count comparison
TC-10	Late-arriving event	Policy applied consistently and documented	Aggregation trace

Project Schedule

Two-Semester Capstone Project Schedule

Period	Illustrative activity
Weeks 1–3	Problem definition, literature review, scope, logging schema.
Weeks 4–6	Data-generation/collection plan, preprocessing, exploratory analysis.

Period	Illustrative activity
Weeks 7–9	Baseline anomaly models and validation procedure.
Weeks 10–12	Backend integration, persistence, review workflow.
Weeks 13–16	Initial testing, documentation, SDP-I milestone.
Second semester Weeks 1–4	Architecture refinement and model comparison.
Weeks 5–8	Evaluation, threshold sensitivity, error analysis.
Weeks 9–12	Security, deployment, reproducibility, performance testing.
Weeks 13–16	Final report, presentation, verification, and handover.

Relevant Academic Coursework

The following courses are relevant to the knowledge areas applied in this project.

Course	Relevance to proposed topic
Programming Fundamentals	Programming and software construction.
Object Oriented Programming	Application design and modular software development.
Data Structures and Algorithms	Efficient processing and algorithmic reasoning.
Design and Analysis of Algorithms	Complexity and systematic algorithm comparison.
Database Systems I	Data persistence, querying, and schema design.
Operating Systems	Systems behavior and resource context.
Data Communications and Computer Networks	Distributed communication and network context.
Statistics and Probability Theory	Statistical reasoning and model evaluation.
Numerical Computing	Numerical methods and computational analysis.
Artificial Intelligence	AI foundations relevant to anomaly-detection methods.
Senior Design Project I	Recorded 2-credit capstone component.
Senior Design Project II	Recorded 4-credit capstone component.